M&J

Engenharia de Requisitos e Desenvolvimento de Software com IA

Especificação resumida — Etapas 1 a 4

Empresa atendida: M&J Tur Locadora de Veículos Ltda.

Apresentação

O M&J será uma aplicação web local para apoiar o transporte universitário da M&J Tur Locadora de Veículos Ltda., facilitando o cadastro de alunos, a consulta de rotas e horários, a confirmação de presença e a organização das viagens.

Restrições do projeto

- Funcionamento local em navegador, utilizando HTML, CSS e JavaScript.
- Sem banco de dados, serviços em nuvem ou APIs externas; armazenamento via localStorage.
- A equipe deverá revisar os resultados gerados com IA e testar a aplicação.

Etapa 1 — Levantamento de requisitos

Perfil	Objetivo e necessidades
Aluno universitário	Cadastrar dados e endereço; consultar rotas/horários; confirmar ou cancelar dias de uso; consultar avisos e atualizar contato.
Motorista da M&J	Consultar rotas e alunos confirmados; acompanhar paradas; atualizar horários e publicar avisos.

Dados principais

Alunos: nome, CPF, endereço, contato, e-mail, senha, rota/ponto de embarque e dias de uso. Motoristas: e-mail, senha e código de autorização. Transporte: rotas, pontos, horários e confirmações. Avisos: mensagem, autor e data.

Restrições e decisões iniciais

- Os dados ficam no navegador e não são sincronizados entre dispositivos.
- Sem mapas ou APIs, a organização de rotas será simplificada e não garantirá o menor trajeto real.
- A autenticação é apenas didática; utilizar dados fictícios na demonstração.
- A equipe ainda deverá confirmar a regra de organização das paradas e o tratamento de cancelamentos recorrentes.

Etapa 2 — Especificação e regras de negócio

Requisitos funcionais (RF)

Código	Requisito
RF01	Cadastrar alunos com nome, CPF, endereço, contato, e-mail e senha.
RF02	Cadastrar motoristas com e-mail, senha e código de autorização.
RF03	Permitir login/logout e separar funções por perfil.
RF04	Consultar rotas, pontos de embarque e horários.
RF05	Confirmar/cancelar uso recorrente por dia da semana.
RF06	Organizar paradas com base nos endereços e presenças confirmadas.
RF07	Permitir ao motorista consultar passageiros e atualizar horários.
RF08	Permitir publicação de avisos pelo motorista.
RF09	Permitir atualização de endereço e contato do aluno.
RF10	Salvar e recuperar dados no localStorage.

Requisitos não funcionais (RNF)

Código	Requisito
RNF01	Funcionamento local, sem servidor remoto, banco de dados ou APIs externas.
RNF02	Interface intuitiva, responsiva e com mensagens claras.
RNF03	Código organizado em funções/módulos para facilitar manutenção.
RNF04	Validar dados antes de salvar e informar erros de operação.
RNF05	Persistência no mesmo navegador; dados não sincronizados entre dispositivos.
RNF06	Exibir dados pessoais apenas quando necessários.

Regras de negócio (RN)

Código	Regra
RN01	Cada conta terá perfil aluno ou motorista; só código correto autoriza cadastro de motorista.
RN02	E-mail único; CPF válido e não duplicado entre alunos.
RN03	Aluno seleciona rota/ponto compatível quando disponíveis e define dias recorrentes.
RN04	A lista de passageiros considera apenas confirmações da viagem correspondente.
RN05	Alterações de endereço ou presença atualizam a organização da rota.
RN06	Motorista visualiza apenas dados necessários; avisos identificam autor.
RN07	Dados são locais e não compartilhados automaticamente entre dispositivos.

Histórias de usuário e critérios essenciais

- HU01 Cadastro: campos válidos salvam; e-mail/CPF duplicados, CPF inválido ou campos vazios são rejeitados.
- HU02 Acesso: credenciais válidas liberam o perfil correto; inválidas exibem “E-mail ou senha inválidos”; logout encerra sessão.
- HU03 Transporte/presença: aluno consulta rotas e horários, seleciona dias e altera/cancela; a lista deve refletir mudanças.
- HU04 Motorista: consulta somente passageiros confirmados, edita horários válidos e publica avisos não vazios com autor/data.
- HU05 Persistência: dados permanecem após recarregar a página; falhas de gravação geram mensagem clara.

Etapa 2 — Validações e limites

Validações e exceções

Situação	Comportamento esperado
Campo obrigatório vazio	Impedir salvamento e indicar o campo.
E-mail/CPF inválido ou duplicado	Solicitar correção ou impedir cadastro.
Senha ou código incorreto	Negar acesso/cadastro e exibir mensagem clara.
Endereço incompleto	Sinalizar e não incluir silenciosamente na rota.
Sem presença confirmada	Exibir mensagem informativa; não gerar paradas de passageiros.
Falha no localStorage	Informar que os dados não puderam ser salvos.
Acesso a função de outro perfil	Negar operação.

Fora do escopo

- GPS em tempo real, cálculo de trajeto com trânsito/ruas reais e integração com mapas.
- Sincronização entre dispositivos, banco de dados, nuvem ou servidor remoto.
- Pagamentos online, notificações externas e recuperação de senha por e-mail.

Decisões pendentes

Tema	Proposta inicial
Organização de rotas	Usar regra local simplificada e declarar a limitação.
Horários	Permitir edição manual pelo motorista.
Cancelamento recorrente	Permitir cancelar um dia específico ou todos os dias, com escolha explícita.
Vínculo motorista/viagem	Identificar um motorista responsável por viagem.
Proteção de dados	Usar dados fictícios e limitar a exibição.

Etapa 3 — Escopo e priorização

Essenciais — MVP

- Cadastro/login de alunos e motoristas com controle de perfil.
- Cadastro de alunos, consulta de rotas e horários.
- Confirmação/cancelamento de presença recorrente.
- Organização simplificada das paradas e consulta de passageiros.
- Atualização de horários pelo motorista.
- Armazenamento local com localStorage.

Desejáveis

- Avisos publicados por motoristas.
- Edição de dados pessoais, interface responsiva e mensagens de sucesso/erro.

Justificativa

A priorização concentra a primeira versão no fluxo principal do transporte e permite implementar e testar as funções dentro do prazo acadêmico, respeitando HTML, CSS, JavaScript e armazenamento local.

Etapa 4 — Planejamento técnico

Persona: Arquiteto de Software. A solução será uma aplicação local dividida em interface, regras de negócio e persistência, sem banco de dados, nuvem ou APIs externas.

Tecnologias

Tecnologia	Responsabilidade
HTML5	Estrutura de páginas, formulários e áreas de consulta.
CSS3	Estilo visual e adaptação a diferentes telas.
JavaScript	Validações, regras de negócio, controle de perfil e atualização das telas.
localStorage	Salvar e recuperar dados no mesmo navegador; não sincroniza dispositivos.

Estrutura de arquivos sugerida

Arquivo/pasta	Responsabilidade
index.html	Entrada da aplicação e estrutura das telas.
css/style.css	Estilos e responsividade.
js/app.js	Inicialização e navegação.
js/auth.js	Cadastro, login, logout e perfis.
js/aluno.js	Dados do aluno e confirmações.
js/motorista.js	Passageiros, horários e avisos.
js/rotas.js	Organização simplificada das paradas.
js/storage.js	Leitura e gravação no localStorage.

Fluxo de dados

1. Ao abrir, consultar as chaves do localStorage; se não houver dados, iniciar com listas vazias ou dados fictícios identificados.
2. Validar entradas antes de alterar dados; depois, converter para JSON e salvar.
3. Se houver erro de leitura/gravação, mostrar mensagem clara.
4. Sugestão de chaves: mj_usuarios, mj_rotas, mj_horarios, mj_confirmacoes e mj_avisos.

Execução e validação

- Salvar os arquivos na estrutura proposta e abrir index.html em navegador atualizado.
- Usar o mesmo navegador para manter os dados locais; evitar modo anônimo na demonstração.
- Se forem usados módulos ES, executar por um servidor local disponível no ambiente da turma.
- Validar campos obrigatórios, formato de e-mail/CPF/horário, duplicidades, perfil de acesso, avisos vazios e estados sem dados.

Limitações e revisão

O localStorage pode ser apagado e não sincroniza dispositivos. A autenticação é didática, não adequada a dados reais. Sem coordenadas ou base geográfica, as paradas serão organizadas por regra simplificada. Antes de implementar, a equipe deverá confirmar que a arquitetura respeita as restrições e testar cada funcionalidade.